

Django Developer – Video Upload Automation via Django Admin & REST API

Goal: Build a Django app that allows uploading videos to multiple third-party social media platforms via **Django Admin** and also provides a **REST API** for automated uploads.

Task Requirements

1. Project Setup

- Create a Django project named `social_video_uploader`.
- Use **Django Admin** for managing video uploads.

2. Models

- `VideoPost` model with fields:
 - Title
 - Description
 - Video file (or URL)
 - Platforms to upload to (multi-select)
 - Upload status (Pending, Success, Failed)

3. Admin Integration

- Admins can create a new `VideoPost` and select platforms.
- Display upload status and timestamps in the admin list view.
- Optional: Filter by platform or status.

4. Third-Party Upload

- Integrate at least **three** of the following platforms (real API or simulated if needed):
 - **Snapshot** – official or simulated API for video posting

- **Vimeo** – Video upload API
- **VK (VKontakte)** – VK API (video upload, live streaming, comments)
- **OK.ru (Odnoklassniki)** – OK API (video upload, sharing, social features)
- **Weibo** – Weibo Open API (supports multimedia posting including video)
- **Twitch** – Twitch API (video uploads / clips)
- **Dailymotion** – Dailymotion API (video upload, publishing)
- **YouTube** – YouTube Data API (video upload, description, title)
- For platforms without public API access, simulate the upload process and update status.

5. REST API

- Provide a **REST endpoint** to:
 - Create a new **VideoPost**
 - Upload a video to selected platforms
 - Return upload status
- Optional: Endpoint to list existing posts and their statuses.

6. Optional Features

- Schedule automatic uploads using a cron job or Celery.
- Log all upload attempts with timestamp, platform, and result.

Deliverables

- Django project with:
 - Working **VideoPost** model and admin interface

- At least **three real or simulated platform integrations**
 - REST API for uploading and listing video posts
 - Screenshots or demo showing admin panel and REST API responses
-

Evaluation Criteria

- Correct Django project structure and admin integration
- Working logic for **multiple platforms**
- REST API functionality with proper responses
- Clear, maintainable, and modular code
- Optional features (scheduling/logging) are a plus